SHOP'IMT : rapport de projet

Projet AWS FISA 2026 · Louison, Gabin et Félix · Oct 30, 2026

Équipe et rendus

SHOP'IMT est la boutique en ligne des étudiants de l'IMT Nord Europe, réalisée pour le projet AWS FISA 2026 (cours Infrastructures cloud et virtualisation, R. Boulkroune) par un groupe de trois, sur un seul compte AWS partagé.

Membre	Parties portées (présentation et rapport)	Accès AWS
Louison	Architecture événementielle, suivi de commande, rôles, solution EC2	Administrateur du compte
Gabin	Exigences du sujet, comparaison serverless / EC2, conclusion et démo	Rôle « stock » (moindre privilège)
Félix	Front bilingue, fiabilité, Terraform et CI/CD, budget	Compte IAM du groupe

Rendu demandé	Où le trouver
4.1 Présentation avec schémas d'architecture	`docs/SHOP-IMT-presentation.pptx` (13 slides, orateur indiqué sur chaque slide)
4.2 Rapport (travail, choix, déroulement)	Ce document, avec en annexe les deux dossiers de conception détaillée (LLD)
4.3 Terraform, lancement et scénarios de test	Dépôt https://github.com/louizoom/shop-imt, section « Lancer Terraform » ci-dessous

Le besoin

Le client veut une boutique en ligne où l'on consulte un catalogue, passe des commandes et reçoit une confirmation en temps réel. La plateforme doit être évolutive, rentable, résiliente et hautement disponible, construite avec des services gérés AWS dans l'offre Free Tier.

- **Clients** : parcourir le catalogue par une interface Web, commander un ou plusieurs produits, recevoir une confirmation.

- **Systèmes internes** : enregistrer toutes les commandes pour l'audit et le reporting, déclencher le traitement interne (emballage, expédition simulée).
- **Architecture** : deux solutions, l'une sur machines virtuelles (EC2), l'autre serverless ; un modèle événementiel SNS et si possible du streaming (Kafka, MSK) ; Terraform pour toute l'infrastructure ; coût maintenu à zéro, quitte à présenter sans l'implémenter une architecture qui dépasserait le budget.

Deux solutions qui tournent en même temps

Les deux solutions servent le même site (Vue 3, français et anglais), la même API et les mêmes e-mails, sur le même compte AWS. Seule l'infrastructure change, ce qui permet de passer une commande sur chacune et de comparer.

```
SERVERLESS   Navigateur → CloudFront → API Gateway → Lambda → DynamoDB →
DynamoDB Streams → λ publication
                                                                    ↓
                          SNS → SQS traitement → Lambda → Step Functions
(expédition simulée)
                              → SQS audit      → Lambda → S3 (audit)
                              → λ notification → SES (e-mail)

EC2          Navigateur → CloudFront → ALB (2 zones) → Auto Scaling 1 à 2 ×
EC2 web → RDS MySQL (privé)
                                                                    ↓
                          SNS → SQS → serveur de traitement EC2 → SES
(confirmation)
                          Kafka (sur ce serveur) → groupe audit → S3 ; groupe
e-mails de suivi → SES
```

	Solution 1 : serverless	Solution 2 : machines virtuelles
Prod	https://shopimt.prodhommed.fr	https://d16dvaumexekob.cloudfront.net
Dev	https://d2953k75zk1mhb.cloudfront.net	https://dwpgpgkoyjtum.cloudfront.net
Calcul	Lambda (Python 3.12, arm64)	EC2 t3.micro, Flask + gunicorn
Données	DynamoDB	RDS for MySQL 8.4
Événements	SNS, SQS, DynamoDB Streams	SNS, SQS, Apache Kafka
Expédition simulée	Step Functions	Serveur de traitement
Code Terraform	`infra/modules/boutique`	`infra/modules/ec2`

La solution EC2 s'active par une seule clé (`solution_ec2`) dans le paramètre SSM de chaque environnement : on l'allume pour la démo et on la coupe ensuite sans toucher au serverless.

Choix d'architecture et alternatives étudiées

Solution 1 : serverless

Choix	Pourquoi	Alternative écartée
Lambda arm64 (Graviton)	20 % moins cher, démarrage rapide	Lambda x86
DynamoDB en capacité provisionnée faible	Reste dans l'offre gratuite, coût prévisible	On-demand, plus cher à charge régulière
DynamoDB Streams comme « outbox » vers Kafka simulé	Aucun événement perdu entre l'écriture et la publication	MSK Serverless (payant à l'heure)
SNS → SQS (fan-out)	Chaque consommateur a sa file et ses reprises	Appels directs entre Lambdas
Step Functions avec étape Wait	Suivi de livraison sans serveur qui attend	Cron EC2
CloudFront + S3 privé (OAC)	HTTPS, cache, bucket jamais public	Site S3 public
Cognito (groupes admins, ventes, stock)	Comptes et rôles sans code d'authentification	Auth maison

Solution 2 : EC2

Choix	Pourquoi	Alternative écartée
ALB + Auto Scaling 1 à 2 t3.micro	Haute disponibilité sur 2 AZ, montée en charge sur le CPU	Une seule instance
RDS MySQL 8.4 single-AZ, sous-réseaux privés	Base relationnelle gérée, offre gratuite	RDS Multi-AZ ou Aurora (environ 2 fois plus cher)
Kafka 3.9.1 KRaft sur une t3.micro	Un vrai broker Kafka pour quelques centimes par jour	Amazon MSK (au moins 70 $ par mois)

Choix	Pourquoi	Alternative écartée
Pas de NAT Gateway, instances en sous-réseaux publics filtrés	Économie d'environ 35 $ par mois	NAT Gateway
CloudFront devant l'ALB avec en-tête secret	HTTPS sans domaine dédié, ALB inaccessible en direct	ALB exposé
Mêmes modules Python que les Lambdas	Une seule logique métier, deux hébergements	Réécriture complète

Alternatives étudiées mais non retenues

- **Amazon MSK et Kinesis** : adaptés au streaming, mais payants à l'heure même sans trafic.
- **NAT Gateway** : remplacé par des groupes de sécurité stricts et des points de terminaison publics.
- **Route 53** : le domaine prodhommed.fr est déjà chez OVH, un CNAME suffit (certificat ACM en us-east-1).
- **WAF, Shield Advanced** : utiles en production réelle, hors budget étudiant.
- **ECS Fargate ou Elastic Beanstalk** : la consigne demande des VM EC2, nous les gérons nous-mêmes.
- **Athena et QuickSight** : l'export CSV dans S3 et le récapitulatif quotidien suffisent pour l'admin.

Correspondance avec la consigne

Exigence de la consigne	Solution 1 serverless	Solution 2 EC2	Statut
Parcourir un catalogue via une interface Web	Vue 3 sur S3 + CloudFront, API Gateway + Lambda, DynamoDB	Même front, Flask sur EC2 derrière ALB, RDS MySQL	OK
Commander un ou plusieurs produits	Panier, Lambda commandes, numéro SHOP-000042-K7QF	Panier, API Flask, transaction MySQL	OK

Exigence de la consigne	Solution 1 serverless	Solution 2 EC2	Statut
Message de confirmation en temps réel	SNS → SQS → Lambda → SES (e-mail)	SNS → SQS → serveur de traitement → SES	OK
Enregistrer toutes les commandes pour audit et reporting	DynamoDB + archive S3, export CSV, récap 21 h	Topic Kafka, groupe audit → S3, RDS	OK
Déclencher le traitement interne (emballage, expédition simulée)	Step Functions (étapes avec Wait)	Worker Python sur EC2, groupe Kafka emails-suivi	OK
Serverless et Compute EC2	Lambda, API Gateway, DynamoDB	EC2, ALB, Auto Scaling, RDS	OK, les deux tournent en même temps
Modèle événementiel SNS	Oui	Oui	OK
Streaming Kafka ou MSK si possible	DynamoDB Streams (équivalent géré gratuit)	Vrai broker Kafka 3.9.1 sur EC2	OK, MSK présenté sans être déployé
Terraform	Module boutique, envs dev et prod, état S3	Module ec2, mêmes envs	OK
Haute disponibilité	Services régionaux multi-AZ par nature	ALB sur 2 AZ, ASG 1 à 2 instances	OK, RDS single-AZ par budget
Free Tier, coût proche de zéro	Environ 0 $	Environ 0,60 $ par jour (IPv4 publiques, 2e instance)	Serverless OK, EC2 à couper après la soutenance
Travail en groupe de 2 ou 3	Louison, Gabin et Félix sur un seul compte		OK
Rendu 4.1 présentation avec schémas et alternatives	PowerPoint 13 slides, orateur par slide		OK
Rendu 4.2 rapport	Ce document + LLD serverless + LLD EC2 en annexe		OK

Exigence de la consigne	Solution 1 serverless	Solution 2 EC2	Statut
Rendu 4.3 Terraform + lancement + scénarios	Dépôt louizoom/shop-imt, section « Lancer Terraform » ci-dessous		OK

Le seul écart assumé est le coût de la solution EC2 : les adresses IPv4 publiques sont facturées depuis 2024, même dans l'offre gratuite. Nous l'activons pour la démonstration et la coupons ensuite avec le paramètre `solution_ec2=false`.

Services AWS : la liste du cours face à notre usage

Service du cours	Utilisé	Comment
EC2	Oui	Serveurs web (ASG) et serveur de traitement + Kafka, solution 2
Lambda	Oui	API, notifications, traitement, récap admin, solution 1 (arm64, X-Ray)
S3	Oui	Site statique, archive d'audit, exports CSV, état Terraform
RDS	Oui	MySQL 8.4 db.t4g.micro, solution 2
DynamoDB	Oui	Produits, commandes, stock, compteur de numéros, solution 1
VPC	Oui	2 AZ, sous-réseaux publics et privés, groupes de sécurité, sans NAT
CloudFront	Oui	Devant S3 (OAC) et devant l'ALB (en-tête secret)
Route 53	Non, justifié	Domaine chez OVH : CNAME vers CloudFront, certificat ACM
IAM	Oui	Rôles par fonction, groupe shopimt-stock pour Gabin, OIDC GitHub Actions sans clé
KMS	Oui, clés gérées par AWS	Chiffrement SSM SecureString, RDS, SQS, S3. Pas de clé client (1 $ par mois)
CloudTrail	Optionnel	Variable `activer_cloudtrail`, coupée par défaut pour le coût S3

Service du cours	Utilisé	Comment
SNS	Oui	Événement « commande créée » diffusé aux files
SQS	Oui	Une file par consommateur + files d'erreurs (DLQ)
CloudWatch	Oui	Logs, alarmes, métrique CPU de l'Auto Scaling
Cost Explorer	Oui	Tag `Solution` (serverless ou ec2) pour séparer les coûts, alerte AWS Budgets à 1 $

Autres services utilisés : API Gateway, Step Functions, Cognito, SES, Translate, EventBridge Scheduler, Systems Manager (Parameter Store et Session Manager), ACM, Elastic Load Balancing, Auto Scaling, X-Ray, AWS Budgets. Au total, 22 services, tous décrits par Terraform.

Domaines de la certification couverts :

- **Concepts du cloud** : élasticité (Auto Scaling, Lambda), multi-AZ, région eu-west-3 et CloudFront mondial.
- **Sécurité** : moindre privilège IAM, OIDC, chiffrement au repos et en transit, buckets privés.
- **Technologie** : compute, stockage, réseau, bases, messagerie et supervision dans les deux solutions.
- **Facturation** : Free Tier, tags de coût, budget, choix délibérés contre les services payants.

Sécurité, haute disponibilité et budget

Sécurité

- **Aucune clé AWS dans GitHub** : les déploiements passent par OIDC, avec un rôle de lecture pour le plan et un rôle de déploiement protégé par l'approbation de Louison en prod.
- **Moindre privilège** : chaque Lambda et chaque instance a son propre rôle. Gabin n'a accès qu'à la table de stock (groupe IAM shopimt-stock).
- **Rôles applicatifs** : groupes Cognito admins, ventes et stock sur le site.
- **Réseau** : RDS en sous-réseaux privés, ALB accessible uniquement depuis CloudFront (liste de préfixes + en-tête secret), pas de port SSH ouvert (Session Manager).

- **Chiffrement** : HTTPS partout (ACM), données chiffrées au repos (KMS géré par AWS), mot de passe RDS généré par Terraform et rangé en SecureString.

Haute disponibilité

- **Serverless** : Lambda, DynamoDB, SNS, SQS et S3 sont multi-AZ par nature. Les files d'erreurs gardent les messages qui échouent.
- **EC2** : l'ALB répartit sur 2 AZ, l'Auto Scaling remplace une instance malade et monte à 2 au-delà de 60 % de CPU.
- **Limites assumées** : RDS et Kafka sont sur une seule AZ pour rester dans le budget. En production, nous passerions à RDS Multi-AZ et à MSK.

Budget

Poste	Coût estimé
Solution serverless	Environ 0 $ (Free Tier)
Solution EC2 active	Environ 0,60 $ par jour (IPv4 publiques et 2e instance)
NAT Gateway évitée	Environ 35 $ par mois économisés
MSK évité	Au moins 70 $ par mois économisés

Une alerte AWS Budgets à 1 $ prévient l'équipe, et le tag `Solution` permet de lire dans Cost Explorer ce que coûte chaque solution.

Déroulement du projet

Nous avons travaillé de façon itérative, comme le recommande la consigne : une première version qui marche vite, puis des ajouts qui mettent chacun en avant un service AWS.

Date	Étape
26 octobre	Lecture de la consigne, comparaison de plusieurs architectures, choix d'un cœur événementiel SNS → SQS partagé et de Kafka sur EC2 plutôt que MSK
27 octobre	Connexion au compte AWS, première solution serverless déployée, boutique nommée SHOP'IMT, e-mails SES, comptes Cognito, dépôt GitHub, environnements dev et prod, front Vue 3 bilingue

Date	Étape
28 octobre	Intégration continue (tests, terraform plan), déploiement par GitHub Actions avec OIDC
29 octobre	Rôles admins, ventes et stock, domaine shopimt.prodhommed.fr, numéros de commande lisibles, présentation de 11 slides
30 octobre	Solution EC2 en parallèle (ALB, Auto Scaling, RDS, Kafka), corrections Kafka, mise en prod EC2, présentation de 13 slides, ce rapport

Problèmes rencontrés et solutions

- **Adresse de Kafka vide au démarrage** : le jeton de métadonnées de l'instance expirait pendant l'installation. Nous passons maintenant l'IP fixe depuis Terraform.
- **Premier événement Kafka perdu après une pause** : un défaut de la bibliothèque kafka-python 3.0.12. Le passage à la version 2.3.3 corrige le problème, vérifié sur plusieurs commandes après 7 minutes d'inactivité.
- **SES en mode sandbox** : seules les adresses vérifiées reçoivent les e-mails, la demande de passage en production est à renouveler auprès d'AWS.
- **Coût des IPv4 publiques** : la solution EC2 est activable par un paramètre pour ne la payer que pendant les démonstrations.

Tests réalisés

- Tests automatiques Python (pytest, dont les tests EC2 contre MySQL) et front (Vitest), lancés à chaque pull request.
- Scénario de bout en bout en dev sur les deux solutions : commande, e-mail de confirmation, archive d'audit dans S3, suivi jusqu'à la livraison (SHOP-000001 livrée en 38 secondes sur EC2).

Lancer Terraform et jouer les scénarios de test

Tout est dans le dépôt GitHub louizoom/shop-imt (README, sections « Première installation » et « Scénarios de test »).

1. Installer sur un compte AWS

1. **Socle**, une seule fois avec un compte administrateur : `cd infra/bootstrap`, puis `terraform init && terraform apply`. Il crée le bucket d'état S3 et les rôles GitHub OIDC.
2. **Réglages** : un paramètre SSM `/shopimt/<env>/config` (JSON) avec l'adresse de notification, les admins et `"solution_ec2": true` pour activer la solution 2.
3. **Déploiement** : `cd infra/envs/dev && terraform init && terraform plan -out tfplan && terraform apply tfplan`. Ou bien un push sur la branche `dev`, et un merge de `dev` dans `main` pour la prod (avec approbation).

2. Jouer les scénarios

```
scripts/scenario.sh dev ton.email@exemple.fr       # solution serverless
scripts/scenario-ec2.sh dev ton.email@exemple.fr   # solution EC2
```

Étape	Serverless	EC2
Infrastructure		Instances de l'Auto Scaling group, ALB fermé sans CloudFront (403)
Catalogue	GET /produits (DynamoDB)	GET /produits (RDS MySQL)
Erreurs	Produit inconnu 400, stock insuffisant 409	Idem
Commande	POST /commandes, numéro SHOP-…	Idem, transaction MySQL
Traitement	Suivi jusqu'à LIVREE via Step Functions	Suivi jusqu'à LIVREE via SQS et Kafka
Audit	Fichier dans S3, ligne DynamoDB	Fichiers écrits dans S3 par le groupe Kafka audit
Admin	401 sans jeton, 200 pour un admin	

L'adresse e-mail doit être vérifiée dans SES tant que le compte est en mode sandbox.

3. Tests sans AWS

- `ruff check . && pytest` à la racine (les tests EC2 utilisent une base MySQL, fournie par la CI).
- `cd frontend && npm run lint && npm test && npm run build`.

4. Tout supprimer

`solution_ec2=false` puis `terraform apply` retire la solution EC2 seule. `terraform destroy` dans chaque environnement supprime tout.

Annexes

- **Annexe A, LLD de la solution serverless** : conception détaillée (API, tables DynamoDB, flux SNS et SQS, Step Functions, e-mails, Cognito). SHOP'IMT serverless : LLD
- **Annexe B, LLD de la solution EC2** : VPC, ALB, Auto Scaling, RDS, Kafka, coût et haute disponibilité. SHOP'IMT EC2 : LLD
- **Présentation** : docs/SHOP-IMT-presentation.pptx, 13 slides, un orateur par slide.
- **Code et Terraform** : dépôt GitHub louizoom/shop-imt.

Dans la version PDF de ce rapport, les deux LLD suivent directement ce document.

SHOP'IMT serverless : LLD

Oct 27, 2026 · Louison, Gabin et Félix

La solution serverless de SHOP'IMT est déployée et testée en eu-west-3. Une commande passe en environ 30 secondes de « reçue » à « livrée », avec un incident de transport tiré au sort. Le coût reste à 0 € dans le Free Tier.

Objet et périmètre

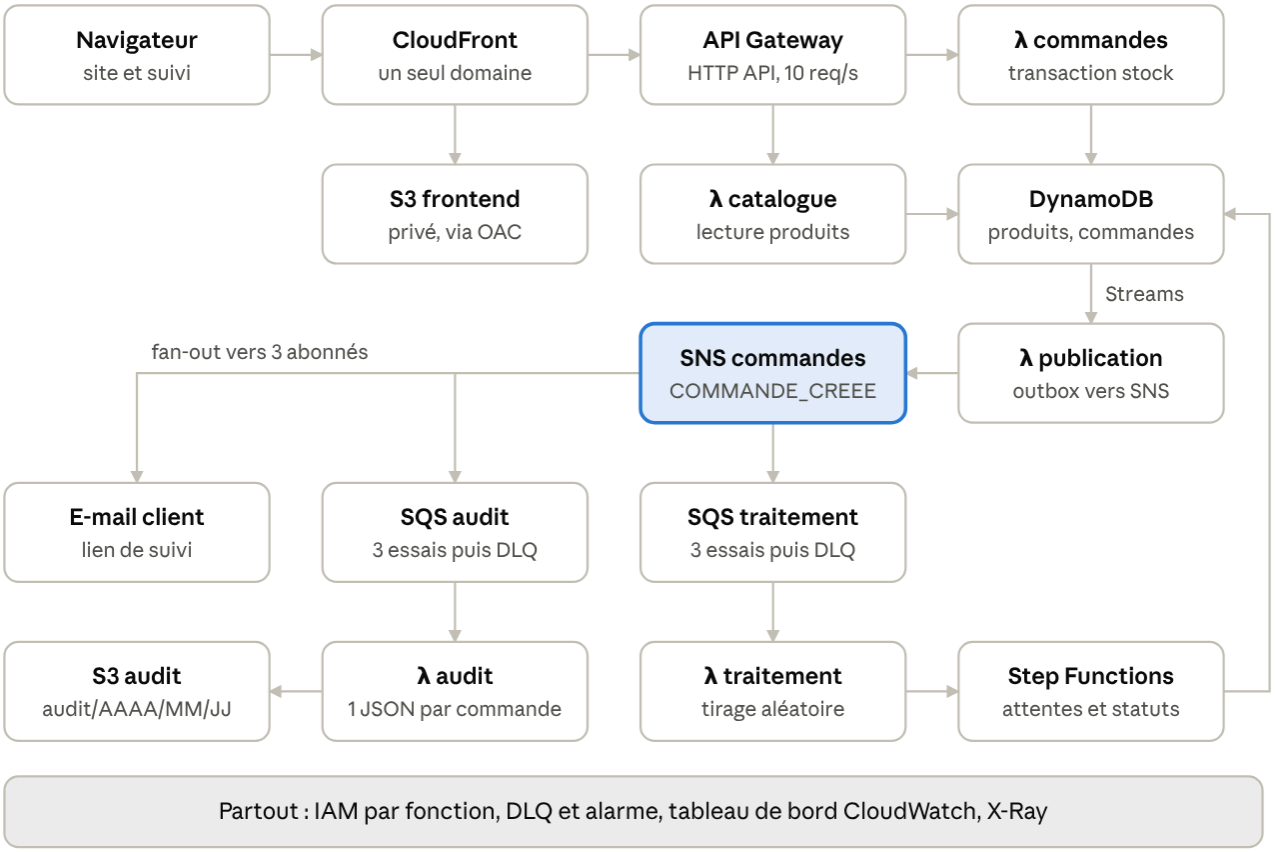
Ce document décrit en détail la **solution 1 (serverless)** de la boutique SHOP'IMT : ce qui est déployé, comment chaque pièce fonctionne, pourquoi ces choix ont été faits, et dans quel ordre le travail a été mené.

Le sujet (projet AWS FISA, R. Boulkroune) demande une boutique en ligne avec un catalogue, des commandes, une confirmation en temps réel, un audit des commandes et un traitement interne simulé. Il exige aussi deux solutions (VM et serverless), un modèle événementiel SNS, du streaming si possible, Terraform et un coût nul.

- **Compte et région** : 204924594218, eu-west-3 (Paris).
- **Site** : prod https://d1rpo2pqzpwl0x.cloudfront.net · dev https://d2953k75zk1mhb.cloudfront.net
- **Code** : dépôt GitHub https://github.com/louizoom/shop-imt (branches dev et main), et archive shop-imt.zip dans le dossier aws du projet.
- **Solution 2** : la boutique sur EC2 (ALB, Auto Scaling, RDS, Kafka) tourne en parallèle de celle-ci et a son propre LLD, annexe B du rapport de projet. Elle reprend les mêmes statuts et le même événement de commande pour que les deux soient comparables.

Vue d'ensemble de l'architecture

Une commande écrite déclenche tout le reste par événements



architecture serverless · 16 composants

La ligne du haut est synchrone : le client attend la réponse. Tout ce qui est sous la table DynamoDB est asynchrone et part du flux des nouvelles commandes, avec SNS au centre.

Service	Rôle dans SHOP'IMT
CloudFront	Entrée HTTPS unique (site et API), cache en edge, en-têtes de sécurité
S3 frontend	`index.html` et `app.js`, privé, lu seulement par CloudFront
API Gateway (HTTP API)	3 routes, limitation de débit, logs d'accès
Lambda (5 fonctions)	catalogue, commandes, publication, traitement, audit
DynamoDB	Tables produits et commandes, transactions

Service	Rôle dans SHOP'IMT
DynamoDB Streams	Flux des nouvelles commandes (le streaming du sujet)
SNS	Topic commandes : 2 files et l'e-mail du client
SQS	Files traitement, audit et DLQ
Step Functions	Préparation, expédition et livraison simulées
S3 audit	Une trace JSON par commande, rangée par date
CloudWatch, X-Ray	Logs, alarme, tableau de bord, traces
IAM, Budgets	Rôles par fonction, alerte de dépense

Parcours d'une commande

Une commande traverse 10 étapes. Seules les deux premières sont synchrones (le client attend la réponse) ; tout le reste est asynchrone et piloté par des événements.

1. **Chargement du site.** Le navigateur ouvre l'adresse CloudFront. CloudFront sert `index.html` et `app.js` depuis le bucket S3 privé (mis en cache 60 s).
2. **Catalogue.** `app.js` appelle `GET /produits` sur le même domaine. CloudFront répond depuis son cache (5 s) ou transmet à API Gateway, qui appelle la λ catalogue (lecture de la table produits).
3. **Commande.** Le client choisit ses articles, saisit son e-mail et valide : `POST /commandes` avec l'e-mail et la liste `{id, quantite}`.
4. **Validation et transaction.** La λ commandes vérifie l'e-mail, 1 à 10 lignes et des quantités de 1 à 20. Elle relit les prix en base, car le prix envoyé par le navigateur n'est jamais utilisé. Une **transaction DynamoDB** décrémente chaque stock (seulement s'il suffit) et écrit la commande au statut `RECUE`. Si un stock manque, rien n'est écrit et l'API répond 409 ; sinon 201 avec le numéro de commande.
5. **Streaming.** L'écriture de la commande apparaît dans le flux **DynamoDB Streams**. Un filtre ne laisse passer que les insertions. La λ publication reconstruit l'événement `COMMANDE_CREEE` et le publie sur SNS.
6. **Diffusion SNS (fan-out).** Le topic livre en parallèle l'événement aux files `traitement` et `audit`, et un texte lisible à l'abonné e-mail dont l'adresse correspond. L'e-mail contient le lien de suivi `/?commande=<id>`.
7. **Traitement.** La λ traitement lit la file et vérifie que la commande est encore `RECUE`. Elle tire au sort un transporteur, un numéro de suivi, la durée de chaque étape (1 à 8 s) et, une fois sur quatre, un incident. Elle lance ensuite une exécution Step Functions qui

porte le numéro de la commande.

8. **Expédition simulée.** Step Functions attend chaque délai, puis met à jour le statut dans DynamoDB : `EN_PREPARATION`, `EMBALLEE`, `EXPEDIEE`, parfois `INCIDENT_TRANSPORT`, `EN_LIVRAISON`, `LIVREE`. Chaque passage ajoute une ligne datée à l'historique.
9. **Audit.** En parallèle, la λ audit écrit l'événement enrichi dans S3, sous `audit/AAAA/MM/JJ/<id>.json`.
10. **Suivi.** La page de suivi interroge `GET /commandes/{id}` toutes les 2 s et affiche la frise, le transporteur, le numéro de suivi et les services AWS actifs, jusqu'à `LIVREE`.

Mesuré le 27/10 à 09h27 : commande reçue à 09:27:00, livrée à 09:27:32 après un incident « Colis retardé au centre de tri », transporteur Chronopost.

Conception détaillée par composant

Chaque composant fait une seule chose. Ils ne se parlent qu'à travers des événements ou des tables, ce qui permet d'en changer un sans toucher aux autres.

CloudFront

- Deux origines : le bucket S3 (Origin Access Control, signé en SigV4) et API Gateway (HTTPS seulement, TLS 1.2).
- `/` vers S3, avec la politique de cache `CachingOptimized`. `/produits` vers l'API avec un cache maison de 5 s. `/commandes*` vers l'API sans cache, en transmettant le corps, la query et les en-têtes (`AllViewerExceptHostHeader`).
- Politique `SecurityHeadersPolicy` (HSTS, anti-iframe, nosniff), HTTP/2 et HTTP/3, `PriceClass_100`.
- Pourquoi : un seul domaine pour le site et l'API, donc pas de CORS et pas d'URL d'API à injecter dans le front.

API Gateway (HTTP API)

L'étape `$default` se déploie automatiquement. Elle est limitée à 10 requêtes/s (rafale de 20) et ses accès sont journalisés en JSON dans CloudWatch.

Route	Lambda	Réponses
`GET /produits`	catalogue	200 : liste `{id, nom, prix, stock}`
`POST /commandes`	commandes	201 : `{order_id, statut, total, articles}` ; 400 : validation ou produit inconnu ; 409 : stock insuffisant

Route	Lambda	Réponses
`GET /commandes/{id}`	commandes	200 : statut, historique, transporteur, numéro de suivi ; 400 : identifiant invalide ; 404 : inconnue

Fonctions Lambda

Toutes tournent en Python 3.12 sur arm64 (Graviton), avec 128 Mo de mémoire et X-Ray actif. Leurs logs JSON sont gardés 7 jours, et elles gèrent les échecs message par message (`ReportBatchItemFailures`).

Fonction	Déclencheur	Droits IAM	Points clés
catalogue	API	`Scan` sur produits	Lecture paginée, tri par id
commandes	API	`BatchGetItem` , `UpdateItem` sur produits ; `PutItem` , `GetItem` sur commandes	Transaction, prix côté serveur, aucun appel SNS
publication	DynamoDB Streams (insertions)	lecture du flux, `sns:Publish` , envoi vers la DLQ	Message double (JSON pour les files, texte pour l'e-mail), s'arrête au premier échec pour garder l'ordre
traitement	SQS traitement, lots de 5	`GetItem` , `UpdateItem` sur commandes ; `states:StartExecution`	Tirage au sort, idempotence par nom d'exécution
audit	SQS audit, lots de 10	`s3:PutObject` sur `audit/*`	Clé rangée par date

Modèle de données DynamoDB

La table **produits** a pour clé `id` et contient `nom` , `prix` et `stock` . Ses 6 produits initiaux sont créés par Terraform, qui ne remet pas le stock à zéro à chaque apply (`ignore_changes`).

La table **commandes** a pour clé `order_id` , avec un flux en `NEW_IMAGE` :

Attribut	Contenu	Écrit par
order_id	UUID v4	commandes

Attribut	Contenu	Écrit par
client_email	adresse en minuscules	commandes
articles	JSON des lignes, prix unitaire figé	commandes
total	nombre	commandes
statut	`RECUE` puis les étapes suivantes	commandes, puis Step Functions
cree_le	date ISO 8601 UTC	commandes
historique	liste de `<date> <STATUT> | <détail>`	commandes, puis Step Functions
transporteur, numero_suivi	texte	traitement

Événement COMMANDE_CREEE

```
{"type": "COMMANDE_CREEE", "order_id": "…", "client_email": "…",
 "articles": [{"id": "p001", "nom": "T-shirt IMT", "quantite": 2,
"prix_unitaire": "19.90"}],
 "total": "49.30", "cree_le": "2026-10-27T09:27:00+00:00"}
```

Le message SNS porte aussi les attributs `client_email` (qui sert de filtre à l'abonnement e-mail) et `type`.

SNS et SQS

- Topic `shopimt-sls-commandes`. Les files le reçoivent en livraison brute (*raw delivery*). L'abonnement e-mail filtre sur `client_email` et doit être confirmé par son destinataire.
- Les files sont chiffrées (SSE-SQS). `traitement` et `audit` ont un délai de visibilité de 60 s et basculent en `dlq` après 3 réceptions. La DLQ garde les messages 14 jours.
- Seul le topic peut écrire dans les files (condition `aws:SourceArn`).

Step Functions « expedition »

- L'entrée est `{order_id, etapes: [{statut, delai, detail}]}`.
- Un état `Map` joue les étapes une par une. Chaque étape attend son délai (`Wait`), puis appelle directement `dynamodb:updateItem`, sans Lambda, avec 3 essais en cas d'erreur.

- Une commande consomme environ 13 transitions d'état, sur les 4 000 gratuites par mois.
- Pourquoi : attendre ne coûte rien, contrairement à une Lambda qui dort, et l'exécution se lit comme un graphe dans la console, ce qui est idéal pour la démo.

Stockage S3

- `shopimt-sls-frontend-<compte>` : privé, lisible seulement par la distribution CloudFront.
- `shopimt-sls-audit-<compte>` : privé, les traces `audit/` expirent après 90 jours.
- Les deux sont en `force_destroy`, pour qu'un `terraform destroy` les vide et les supprime.

Supervision

- L'alarme `shopimt-sls-dlq-non-vide` se déclenche dès qu'un message arrive en DLQ.
- Le tableau de bord `shopimt-sls-tableau-de-bord` suit les requêtes, erreurs et latences p50/p95 de l'API, les invocations et erreurs Lambda, les files en attente, les publications SNS et la capacité DynamoDB consommée.
- X-Ray montre la carte des services et le temps passé dans chaque appel AWS.

Sécurité, résilience, coût

Sécurité

- **Moindre privilège** : un rôle IAM par Lambda et un pour Step Functions, limités aux actions et aux ressources nécessaires.
- **Rien de public** : les buckets sont privés, et le site n'est lisible qu'à travers CloudFront.
- **Chiffrement** : HTTPS de bout en bout. Les données sont chiffrées au repos dans DynamoDB, S3 et SQS, avec des clés gérées par AWS (gratuites).
- **Application** : validation stricte des entrées et prix recalculés côté serveur. Le front n'injecte jamais de HTML, ce qui évite les failles XSS. L'API est limitée en débit, et CloudFront ajoute des en-têtes de sécurité.
- **Déploiement** : le déploiement passe par l'utilisateur IAM dédié `shopimt-deployer`, jamais par le compte root. Sa clé d'accès est créée pour un déploiement puis supprimée aussitôt.

Résilience

- **Haute disponibilité** : tous les services sont régionaux et répartis sur plusieurs zones de disponibilité par AWS. CloudFront sert le site depuis ses points de présence.
- **Cohérence** : la transaction DynamoDB empêche la survente et les commandes à moitié écrites.
- **Pas d'événement perdu** : avec le pattern *outbox*, une commande écrite est forcément publiée, car le flux réessaie jusqu'à 3 fois avant la DLQ.
- **Erreurs isolées** : un message en échec ne bloque pas les autres. Après 3 échecs, il part en DLQ et déclenche l'alarme.
- **Idempotence** : une relivraison ne relance rien. L'exécution Step Functions porte le nom de la commande, le transporteur n'est fixé qu'une fois, et la clé d'audit est l'identifiant de la commande.

Coût

Service	Réglage	Offre gratuite utilisée
Lambda	128 Mo, arm64, aucune attente active	1 M d'appels et 400 000 Go-s par mois
DynamoDB	provisionné 5/5 par table, flux	25 RCU / 25 WCU / 25 Go ; lecture du flux par Lambda gratuite
Step Functions	Standard, états Wait	4 000 transitions par mois
SNS, SQS	standard	1 M de requêtes par mois chacun, 1 000 e-mails
CloudFront	PriceClass_100	1 To et 10 M de requêtes par mois
API Gateway	HTTP API	offre gratuite ou crédits du compte
CloudWatch	logs 7 j, 1 alarme, 1 tableau de bord	5 Go de logs, 10 alarmes, 3 tableaux de bord
X-Ray	Lambdas et Step Functions	100 000 traces par mois

Ne sont pas déployés, car payants : MSK, Kinesis, WAF, clés KMS client, Athena, Route 53 et NAT Gateway. CloudTrail est prêt mais désactivé par défaut (`activer_cloudtrail`), car son stockage S3 coûterait quelques centimes par mois.

Correspondance avec le sujet

Toutes les exigences du sujet sont couvertes pour la partie serverless. Le streaming, demandé « si possible », passe par DynamoDB Streams ; la solution EC2, déployée en parallèle, utilise un vrai broker Kafka.

Exigence du sujet	Réponse dans SHOP'IMT
Parcourir un catalogue via une interface Web	Site S3 + CloudFront, `GET /produits`
Commander un ou plusieurs produits	`POST /commandes`, transaction DynamoDB sur le stock
Recevoir une confirmation en temps réel	Réponse immédiate, e-mail SNS avec lien de suivi, page de suivi en direct
Enregistrer les commandes pour audit et reporting	SQS audit vers S3, rangé par date, prêt pour Athena
Déclencher emballage et expédition simulés	SQS, Lambda puis Step Functions, avec aléatoire et incidents
Services serverless	Lambda, API Gateway, DynamoDB, SNS, SQS, Step Functions, S3, CloudFront
Modèle événementiel SNS	Un topic, 3 abonnés, filtre sur l'e-mail
Streaming (Kafka, MSK) si possible	DynamoDB Streams ; Kafka 3.9.1 sur EC2 dans la solution 2 ; MSK présenté en architecture cible car payant
Terraform	100 % de l'infrastructure, données initiales comprises
Free Tier, coût zéro	Réglages détaillés dans la partie coût
Haute disponibilité	Services managés multi-AZ, CloudFront
README pour lancer et tester	README, tests unitaires et `scenario.sh`
Solution sur machines virtuelles (EC2)	Solution 2 en ligne depuis le 30 octobre (LLD EC2)

Journal de réalisation

Le travail s'est fait en 8 étapes, du 26 au 27 octobre. Chacune a été vérifiée avant de passer à la suivante.

Date	Étape	Ce qui a été fait, et pourquoi
27/10, 09h28	Redéploiement des améliorations	10 ressources ajoutées, 14 modifiées, 1 supprimée. Commande de test livrée en 32 s avec un incident ; DynamoDB, audit S3 et Step Functions vérifiés, DLQ vide. Clé de déploiement supprimée, nouvel état et nouveau zip (43 Ko) sauvegardés.
27/10, 08h53	Améliorations demandées par Louison	Streaming DynamoDB Streams et λ publication (outbox). API servie par CloudFront, cache du catalogue, en-têtes de sécurité. Expédition aléatoire avec Step Functions et statut `LIVREE`. Lien de suivi dans l'e-mail et page de suivi. Refonte du site, testée avec Playwright sur une fausse API (clair, sombre, mobile). Tableau de bord CloudWatch, X-Ray, CloudTrail en option. Tests unitaires étendus.
27/10, 08h46	Premier déploiement	La clé AWS du poste de Louison était invalide, donc le déploiement s'est fait depuis l'environnement de Claude avec l'utilisateur dédié `shopimt-deployer`. 57 ressources créées. Scénario réussi : erreurs 400 et 409, commande expédiée, audit S3.
27/10, 08h33	Vérification du livrable	Tests unitaires OK. `terraform validate` OK avec le provider AWS 5.80 et 6.70. Corrections : formatage, provider borné à `< 7.0`, budget désactivé par défaut puisque le compte en avait déjà deux.
27/10, matin	Renommage en SHOP'IMT	Fait par Louison dans tout le livrable.
27/10, matin	Première version serverless	4 Lambdas, API Gateway, DynamoDB, SNS, 2 files SQS, S3, CloudFront, DLQ, alarme, budget. Tests unitaires avec un faux SDK AWS.
27/10, 07h58	Connexion du compte AWS	Connecteur AWS branché, ressources de test `boutique-test-*`, budget à 1 $, comptes IAM pour Gabin et Félix.

Date	Étape	Ce qui a été fait, et pourquoi
26/10	Pistes d'architecture	3 pistes comparées. Recommandation : un cœur événementiel SNS → SQS commun aux deux solutions, avec Kafka sur une petite instance EC2 plutôt que MSK pour la solution 2.

Mise à jour du 27/10, 09h44 : identité visuelle. Le site reprend les couleurs de la charte IMT (bleu IMT #00B8DE, bleu marine #14223C, gris #EDF3F4) et la typographie Helvetica. Il a un logo SHOP'IMT original et un visuel SVG détaillé pour chacun des 6 articles (36 Ko au total), publiés par Terraform dans `img/`. Depuis, le logo complet IMT Nord Europe figure dans la barre de navigation, sur les photos des produits et en bas des e-mails.

Déployer, tester, démontrer, supprimer

Déployer

```
# Développement : une branche depuis dev, une pull request vers dev
git switch dev && git switch -c ma-fonctionnalite
# ... modifications, puis pull request : la CI teste et affiche le terraform
plan de dev et prod

# Fusion dans dev  -> GitHub Actions déploie l'environnement dev
# Fusion de dev dans main -> GitHub Actions déploie la prod (validation
possible)

# À la main, depuis un poste avec des identifiants AWS :
cd frontend && npm ci && npm run build && cd ..
cd infra/envs/dev
terraform init
terraform plan -out tfplan
terraform apply tfplan
```

Le front doit être compilé (`npm run build`) avant le plan : Terraform publie le dossier `frontend/dist` dans S3.

L'état Terraform de chaque environnement est dans S3 (bucket `shopimt-tfstate-204924594218` , clés `dev/terraform.tfstate` et `prod/terraform.tfstate`), avec un verrou natif S3 : deux personnes ne peuvent pas appliquer en même temps, et tout le monde travaille sur le même état. L'ancien dossier `aws/shop-imt-serverless-etat` est obsolète, il ne faut plus s'en servir.

Tester

- `ruff check . && pytest` : tests sans AWS (validation, stock insuffisant et annulation, publication par le flux, tirage aléatoire, idempotence, audit).
- `scripts/scenario.sh dev ton.email` : scénario de bout en bout. Il passe par le catalogue, deux erreurs (400, 409) et une commande suivie jusqu'à `LIVREE`, puis vérifie DynamoDB, l'audit S3 et l'exécution Step Functions.

Démontrer en 5 minutes

1. Commander deux articles sur le site : la page de suivi s'ouvre et avance toute seule.
2. Ouvrir l'e-mail de confirmation et cliquer sur le lien de suivi.
3. Dans la console : le graphe de l'exécution Step Functions, le fichier d'audit dans S3, le tableau de bord CloudWatch et la carte X-Ray.
4. Demander plus de gourdes qu'il n'y en a en stock : refus immédiat, et le stock ne bouge pas.

Supprimer

`terraform destroy` dans infra/envs/dev ou infra/envs/prod (chaque environnement se supprime séparément). Les buckets sont vidés automatiquement. Le socle (infra/bootstrap) est protégé contre la suppression, car il garde l'état des deux environnements.

Limites et pistes

- **Rejouer l'historique des événements** : DynamoDB Streams garde 24 h ; MSK Serverless ou Kinesis garderaient tout (payants).
- **Vrais e-mails clients** : SES, car SNS impose que chaque adresse confirme son abonnement.
- **Comptes clients** : Cognito et une autorisation JWT, pour que seul le client voie sa commande. Aujourd'hui, c'est le numéro de commande aléatoire qui sert de secret.
- **Reporting** : Athena puis QuickSight sur le bucket d'audit.
- **Protection** : WAF devant CloudFront (payant).

Mise à jour : e-mail HTML (SES), comptes clients (Cognito) et version mobile

E-mail de confirmation avec Amazon SES

SNS ne sait envoyer que du texte brut. L'abonnement e-mail SNS a donc été remplacé par un abonné Lambda, `notification`, qui construit un e-mail HTML et l'envoie avec **Amazon SES v2** (`ses:SendEmail`).

- Contenu : logo SHOP'IMT, photo de chaque article (PNG servis par CloudFront, car les messageries n'affichent pas le SVG), quantités, total, bouton « Suivre ma commande → » vers `/?commande=<id>`, et une version texte de secours.
- Sécurité : tous les champs sont échappés (`html.escape`), testé contre l'injection de HTML.
- Terraform : `aws_sesv2_email_identity` pour l'expéditeur et pour chaque adresse de `emails_verifies`.
- Sandbox SES : seules les adresses vérifiées reçoivent l'e-mail. Un destinataire non vérifié est journalisé (`EMAIL_REFUSE`) puis ignoré, sans relance. Vérifié le 27/10 : l'e-mail de la commande `c67ccf7c` a été envoyé (`EMAIL_ENVOYE`).

Comptes clients avec Amazon Cognito

1. **User pool** `shopimt-sls-clients` : inscription par e-mail, code de confirmation envoyé par Cognito, mot de passe de 8 caractères minimum.
2. **Client public** `shopimt-sls-site` (sans secret) avec les flux `USER_PASSWORD_AUTH` et `REFRESH_TOKEN_AUTH`. Le site appelle directement l'API Cognito (`SignUp`, `ConfirmSignUp`, `InitiateAuth`).
3. **Authorizer JWT** sur l'API HTTP : les routes `POST /compte/commandes` et `GET /compte/commandes` refusent toute requête sans jeton valide (401), sans écrire une ligne de code de vérification.
4. **DynamoDB** : index secondaire global `par-client` (`client_sub` + `cree_le`) sur la table des commandes. « Mes commandes » est une simple requête sur cet index, la plus récente en premier.
5. **CloudFront** : comportement `/compte/*` sans cache, avec l'en-tête `Authorization` transmis à l'API.
6. **config.json** publié sur S3 par Terraform (région et identifiant du client Cognito) : le site n'a aucune valeur codée en dur.

Une commande passée connecté prend l'e-mail du jeton (il ne peut pas être falsifié) et enregistre `client_sub`. Test réel du 27/10 : 401 sans jeton, puis commande créée et retrouvée dans l'historique avec le jeton.

Version mobile (iPhone)

Sur téléphone, la navigation passe dans une barre d'onglets en bas (Boutique, Suivre, Mon compte), avec un panier flottant qui affiche le nombre d'articles et le total. Le site respecte les zones de l'encoche et de la barre d'accueil (`viewport-fit=cover` , `safe-area-inset`), agrandit les boutons à 44 px, et peut être ajouté à l'écran d'accueil (icône et `manifest.webmanifest`). Ce sont des changements HTML/CSS : ils n'ajoutent aucun service AWS.

Interface admin

Un onglet « Admin », visible seulement pour les comptes du groupe Cognito `admins` , affiche les commandes avec l'e-mail du client et les statistiques sur 7, 30 ou 90 jours. Thème sombre et rouge, distinct de la boutique.

1. Le front lit le groupe dans le jeton d'identité Cognito (`cognito:groups`) pour afficher l'onglet.
2. `GET /admin/stats` et `GET /admin/commandes` passent par CloudFront (`/admin/*` , sans cache) et l'autorisateur JWT d'API Gateway : 401 sans jeton.
3. La Lambda `admin` vérifie le groupe (403 sinon) et lit l'index secondaire `par-jour` (clé `jour` , tri `cree_le`) : une requête Query par jour, jamais de Scan. Son rôle IAM n'autorise que cette lecture.
4. Elle calcule commandes, chiffre d'affaires, panier moyen, clients uniques, répartition des statuts et meilleures ventes. `GetMetricData` n'est pas utilisé car il est facturé dès le premier appel.

La Lambda `publication` écrit aussi deux métriques métier, `Commandes` et `ChiffreAffaires` , au format EMF (Embedded Metric Format) : une ligne de journal que CloudWatch convertit en métrique, sans appel d'API. Elles s'affichent dans le tableau de bord CloudWatch. Les admins sont déclarés dans la variable Terraform `admins` (comptes déjà inscrits sur le site).

Export CSV. La route `GET /admin/export` écrit un fichier CSV dans le bucket d'audit S3 (préfixe `exports/` , supprimé après 1 jour par une règle de cycle de vie) et renvoie une URL présignée valable 5 minutes. Le bucket reste privé.

Récap quotidien des ventes

Amazon EventBridge Scheduler déclenche chaque soir à 21 h (Europe/Paris) la Lambda `rapport` . Elle lit les commandes du jour par l'index `par-jour` et les stocks inférieurs ou égaux à 5, puis envoie un e-mail HTML via SES aux adresses de `admins` . Le planificateur a son propre rôle IAM, limité à `lambda:InvokeFunction` sur cette fonction. Premier envoi vérifié le 27 octobre 2026 (28 commandes, 1 691,40 €).

Service AWS ajouté	Usage	Offre gratuite
Cognito (groupes)	Groupe `admins`	Inclus
DynamoDB (index `par-jour`)	Lecture des commandes par date	20 RCU / 20 WCU utilisés sur 25
CloudWatch (EMF)	2 métriques métier	10 métriques
EventBridge Scheduler	1 déclenchement par jour	14 M d'invocations / mois

Services AWS ajoutés et coût

Service	Rôle	Gratuité
Amazon SES	E-mail HTML de confirmation	3 000 messages / mois pendant 12 mois
Amazon Cognito	Comptes clients, jetons JWT	10 000 utilisateurs actifs / mois
API Gateway (authorizer JWT)	Protection des routes du compte	inclus dans l'API HTTP
DynamoDB (index secondaire)	Historique par client	dans les 25 RCU / WCU gratuits

Mise à jour : dépôt GitHub, environnements dev et prod, front Vue bilingue

Le projet est passé d'un dossier unique à un dépôt organisé comme un projet professionnel : deux environnements identiques, un état Terraform partagé, une intégration continue qui affiche le plan Terraform à chaque modification, et un front réécrit en Vue 3 + TypeScript, en français et en anglais.

Organisation du dépôt

Dossier	Contenu
`backend/lambdas`	Code des fonctions Lambda (Python 3.12), une par fichier

Dossier	Contenu
`backend/tests`	Parcours complet avec un faux boto3 (pytest), sans appel à AWS
`frontend`	Vue 3, TypeScript, Vite, Vue Router, vue-i18n, ESLint, Prettier, Vitest
`infra/modules/boutique`	Toute l'infrastructure, paramétrée (préfixe, capacités DynamoDB, identités SES)
`infra/envs/dev` , `infra/envs/prod`	Un fichier par environnement qui appelle le module avec ses réglages
`infra/bootstrap`	Socle créé une seule fois : bucket d'état et rôles GitHub
`.github/workflows`	CI (`ci.yml`) et déploiement (`deploy.yml`)

Environnements dev et prod

Les deux environnements utilisent exactement le même module Terraform : une modification testée en dev est donc la même que celle qui partira en prod.

	dev	prod
Branche Git	`dev`	`main`
Préfixe des ressources	`shopimt-dev`	`shopimt-sls`
Site	https://d2953k75zk1mhb.cloudfront.net	https://d1rpo2pqzpwl0x.cloudfront.net
Capacité DynamoDB	1 RCU / WCU par table et index	5 RCU / WCU
Identités SES	partagées avec la prod	domaine et adresses vérifiées
Comptes clients	pool Cognito séparé	pool Cognito de prod

Avec 2 tables et 2 index, la capacité totale est de 24 unités, sous les 25 gratuites. Les adresses e-mail personnelles (expéditeur, admins) ne sont pas dans le dépôt : chaque environnement les lit dans un paramètre **SSM Parameter Store** (`/shopimt/<env>/config`).

État Terraform et socle

- L'état est dans un bucket **S3** versionné, chiffré et privé, avec le verrou natif de S3 (`use_lockfile` , sans table DynamoDB).

- La prod existante a été migrée sans rien recréer : ses 155 adresses ont été déplacées sous `module.boutique`, et le plan suivant affichait « No changes ».
- Le lock file Terraform contient les empreintes des providers pour Linux, macOS et Windows : chaque membre du groupe peut faire `terraform init` sur son poste.

CI/CD avec GitHub Actions

- À chaque pull request : `ruff` et `pytest` pour les Lambdas ; lint, tests, vérification des types et build pour le front ; `terraform fmt` et `validate`, puis le **plan de dev et de prod** affiché dans le résumé du workflow.
- À chaque push sur `dev` : déploiement de l'environnement dev. À chaque push sur `main` : déploiement de la prod, avec validation par un membre si l'environnement GitHub `prod` l'exige.
- **Aucune clé AWS n'est stockée dans GitHub.** GitHub Actions s'authentifie par **OIDC** auprès d'IAM et reçoit des identifiants temporaires d'une heure. Le rôle `shopimt-github-plan` est en lecture seule ; le rôle `shopimt-github-deploy` n'est accessible qu'aux jobs des environnements GitHub `dev` et `prod` du dépôt.
- Dependabot propose une fois par mois, vers `dev`, la mise à jour des dépendances.

Front Vue 3 + TypeScript

- Pages : boutique, suivi de commande, compte client, console admin (ventes et stock). Les types des réponses de l'API sont déclarés en TypeScript.
- Navigation par **Vue Router** : `/suivi/<id>`, `/compte`, `/admin`. La route `/admin` est protégée par le groupe Cognito `admins` (l'API revérifie de toute façon). Les anciens liens `/?commande=…` des e-mails redirigent vers la bonne page.
- Une **CloudFront Function** de routage (viewer-request) sert `index.html` pour toute adresse sans extension : `/suivi/<id>` s'ouvre directement depuis un e-mail.
- Le JavaScript est découpé par page : la console admin n'est chargée que par les administrateurs.

Version française et anglaise

Où	Comment
Textes du site	vue-i18n, un fichier par langue ; un test vérifie que les deux ont les mêmes clés
Langue par défaut	**CloudFront Function** (viewer-response) : d'après `Accept-Language`, elle dépose un cookie `lang` à la première visite

Où	Comment
Catalogue	**Amazon Translate** traduit le nom des produits à la première demande, puis le garde dans DynamoDB (`nom_en`)
E-mails	La commande garde sa langue ; les e-mails SES (confirmation, statut, rappel) partent dans cette langue

Services AWS ajoutés et coût

Service	Rôle	Gratuité
CloudFront Functions	Langue par défaut, routage du front	2 millions d'appels / mois
Amazon Translate	Nom anglais des produits	2 millions de caractères / mois pendant 12 mois
SSM Parameter Store	Réglages personnels hors du dépôt	paramètres standard gratuits
IAM OIDC	Connexion de GitHub Actions sans clé	gratuit
S3 (état Terraform)	État partagé et verrou	quelques Ko, dans les 5 Go gratuits

Le second environnement double les appels, mais l'usage d'un projet étudiant reste dans les quotas gratuits.

Mise à jour du 29/10 : rôles de gestion (Cognito et IAM)

La console de gestion n'est plus réservée aux seuls administrateurs : chaque membre de l'équipe reçoit un rôle qui ne donne que ce dont il a besoin. Le même principe s'applique sur le site (groupes Cognito) et dans la console AWS (groupes IAM). Gabin, qui était administrateur partout, gère désormais uniquement le stock.

Les trois rôles

Rôle	Groupe Cognito (site)	Ce qu'il voit et fait	Pendant IAM (console AWS)
Admin	`admins` (priorité 1)	Ventes, commandes, export CSV, stock	administrateur du compte
Ventes	`ventes` (priorité 10)	Graphiques et commandes, sans export ni stock	aucun
Stock	`stock` (priorité 20)	Onglet Stock seulement (réassort + / −)	groupe `shopimt-stock`

Comment le contrôle est fait

1. **Jeton** : à la connexion, Cognito place les groupes de l'utilisateur dans le jeton d'identité (`cognito:groups`).
2. **API Gateway** : l'autorisateur JWT refuse toute requête `/admin/*` sans jeton valide (401), avant d'appeler la Lambda.
3. **Lambda admin** : une table de droits associe chaque rôle aux routes permises (`GET /admin/stats` , `GET /admin/commandes` , `GET /admin/export` , `GET` et `POST /admin/stock`). Une route non permise renvoie 403 et la tentative est journalisée dans CloudWatch (`ADMIN_REFUSE` , avec l'utilisateur, la route et ses groupes).
4. **Front** : le lien du menu s'appelle Admin, Stock ou Ventes selon le rôle, et seuls les onglets permis s'affichent. C'est du confort : la sécurité repose sur l'API.

Les membres de chaque rôle sont lus dans le paramètre SSM `/shopimt/<env>/config` (clés `admins` , `lecteurs_ventes` , `gestionnaires_stock`) : aucune adresse personnelle dans le dépôt. Un membre doit se déconnecter puis se reconnecter pour recevoir un nouveau rôle.

Moindre privilège dans la console AWS

Le groupe IAM `shopimt-stock` (dans `infra/bootstrap`) remplace, pour Gabin, le groupe administrateur `boutique-admins` . Il autorise :

- la lecture des tables `shopimt-*-produits` (DescribeTable, Scan, Query, GetItem) ;
- `UpdateItem` sur ces tables, avec un **contrôle d'accès fin DynamoDB** : la condition `dynamodb:Attributes` n'accepte que les attributs `id` et `stock` . Modifier un prix ou un nom est refusé ;
- la lecture des journaux de la Lambda admin (pour voir les actions de gestion) ;

- le changement de son propre mot de passe.

Vérifié le 29/10 avec le simulateur de politiques IAM : modifier le stock est autorisé ; modifier le prix, supprimer un produit, créer un utilisateur IAM ou lister les buckets S3 sont refusés.

Déploiement et tests

- Tests : pytest couvre les quatre cas (stock, ventes, admin, et cumul de deux rôles), avec les 403 attendus.
- Déployé le 29/10 en prod et en dev : 3 groupes Cognito, Gabin dans `stock` en prod, nouvelle version de la Lambda admin et du front. Le groupe `admins` existant a été conservé (bloc `moved` Terraform), sans recréation.

Service AWS	Usage	Coût
Cognito (groupes)	3 rôles, priorités	inclus
IAM (groupe + politique en ligne)	Rôle stock dans la console	gratuit
DynamoDB (contrôle d'accès fin)	Écriture limitée à l'attribut stock	gratuit
CloudWatch Logs	Journal des refus `ADMIN_REFUSE`	dans les 5 Go gratuits

SHOP'IMT EC2 : LLD

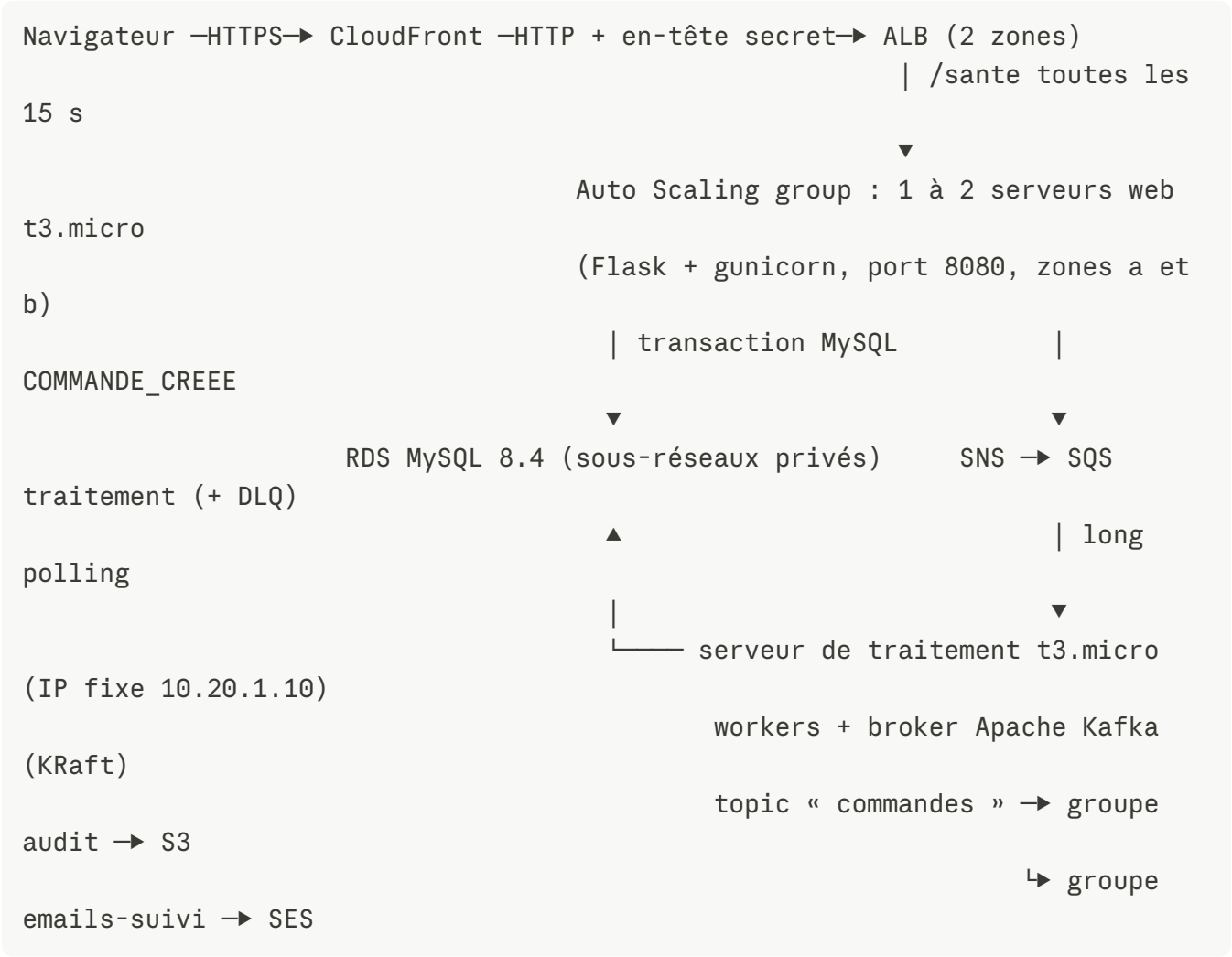
Oct 30, 2026 · Louison, Gabin et Félix

Objet et périmètre

Ce document décrit la **solution 2 (machines virtuelles EC2)** de SHOP'IMT. Elle tourne **en même temps** que la solution serverless, sur le même compte, avec le même front, la même API et les mêmes e-mails : seule l'infrastructure change. On peut ainsi passer une commande sur chacune et comparer.

- **Code** : `infra/modules/ec2` (Terraform) et `backend/ec2` (serveurs Python), dans le dépôt https://github.com/louizoom/shop-imt.
- **Activation** : clé `solution_ec2` du paramètre SSM `/shopimt/<env>/config`. Absente ou `false` : rien n'est créé. Le serverless n'est jamais touché.
- **Réutilisation** : les serveurs importent les modules des Lambdas (validation des commandes, numéros `SHOP-000042-K7QF`, date de livraison, agenda `.ics`, tirage du plan d'expédition, e-mails HTML FR/EN). Une règle métier ne s'écrit qu'une fois.

Vue d'ensemble de l'architecture



Service	Rôle dans la solution EC2
VPC	Réseau dédié `10.20.0.0/16` sur 2 zones : sous-réseaux publics (ALB, serveurs) et privés (RDS, sans route Internet)
Internet Gateway, endpoint Gateway S3	Sortie vers les API AWS sans NAT Gateway ; le trafic S3 reste sur le réseau AWS (gratuit)
EC2 (t3.micro, Amazon Linux 2023)	Serveurs web et serveur de traitement, IMDSv2 obligatoire, disques chiffrés
Auto Scaling	1 à 2 serveurs web répartis sur 2 zones, ajout au-delà de 60 % de CPU, remplacement d'une instance malade, mise à jour progressive
Elastic Load Balancing (ALB)	Répartition de charge, contrôle de santé `/sante`, accepte seulement CloudFront

Service	Rôle dans la solution EC2
CloudFront	HTTPS, cache des fichiers statiques, protection de l'ALB
RDS for MySQL 8.4	Catalogue, commandes, historique ; TLS obligatoire, chiffrement KMS, sauvegarde quotidienne
SNS, SQS	Diffusion de `COMMANDE_CREEE` vers la file de traitement, DLQ après 3 échecs
Apache Kafka sur EC2	Flux de tout le cycle de vie des commandes, deux groupes de consommateurs
SES	E-mails de confirmation et de suivi (même domaine vérifié que le serverless)
SSM	Mot de passe RDS en SecureString, Session Manager à la place de SSH, AMI la plus récente
CloudWatch	Agent (logs, mémoire), alarmes, récupération automatique, tableau de bord
IAM	Un rôle par type de serveur (profil d'instance), aucune clé d'accès

Parcours d'une commande

1. **Page et catalogue.** Le navigateur ouvre l'adresse CloudFront. Les fichiers `/assets/*` et `/img/*` sont servis depuis le cache de CloudFront ; le reste passe par l'ALB jusqu'à un serveur web, qui lit `produits` dans RDS.
2. **Commande.** `POST /commandes` arrive sur l'un des serveurs web. La validation est celle de la Lambda `commandes`.
3. **Transaction MySQL.** `SELECT … FOR UPDATE` verrouille les lignes des produits commandés : deux commandes simultanées, même sur deux serveurs différents, s'attendent. Le stock est vérifié puis décrémenté, la commande et sa première ligne d'historique sont écrites, et le tout est validé en une fois (`COMMIT`). Stock insuffisant : `ROLLBACK` et réponse 409.
4. **Numéro.** La clé `AUTO_INCREMENT` de MySQL donne le numéro `SHOP-000042`, suivi de 4 caractères aléatoires (même format que le compteur DynamoDB du serverless).
5. **Événements.** Le serveur web publie `COMMANDE_CREEE` sur **SNS** (vers la file SQS de traitement) et sur **Kafka** (topic `commandes`, clé = numéro de commande).

6. **Traitement.** Le serveur de traitement lit la file SQS en long polling. Il réserve la commande (verrou de ligne, une seule fois), envoie l'e-mail de confirmation SES, puis joue les étapes tirées au sort : `EN_PREPARATION`, `EMBALLEE`, `EXPEDIEE`, parfois `INCIDENT_TRANSPORT`, `EN_LIVRAISON`, `LIVREE`.
7. **Statuts.** Chaque étape est écrite dans RDS (statut et historique) et publiée sur Kafka.
8. **Audit.** Le groupe Kafka `audit` écrit chaque événement dans S3 : `audit/AAAA/MM/JJ/<commande>/<partition>-<offset>-<statut>.json`.
9. **E-mails de suivi.** Le groupe Kafka `emails-suivi` envoie les e-mails d'expédition, d'incident et de livraison.
10. **Suivi.** La page `/suivi/<numéro>` interroge `GET /commandes/<numéro>` toutes les 2 secondes jusqu'à `LIVREE`.

Conception détaillée

Réseau

Sous-réseau	Zone	Plage	Contenu
public a	eu-west-3a	10.20.1.0/24	ALB, serveurs web, serveur de traitement (10.20.1.10)
public b	eu-west-3b	10.20.2.0/24	ALB, serveurs web
privé a	eu-west-3a	10.20.11.0/24	RDS
privé b	eu-west-3b	10.20.12.0/24	RDS (2e zone exigée par RDS, utilisée par Multi-AZ)

Les groupes de sécurité forment une chaîne : chaque couche n'accepte que la précédente.

Groupe	Entrée autorisée	Sortie autorisée
alb	80 depuis la liste de préfixes CloudFront	8080 vers web
web	8080 depuis alb	443 et 80 (API AWS, paquets), 3306 vers bdd, 9092 vers traitement
traitement	9092 depuis web	443 et 80, 3306 vers bdd

Groupe	Entrée autorisée	Sortie autorisée
bdd	3306 depuis web et traitement	aucune

Aucun port SSH n'est ouvert : l'accès aux serveurs passe par **SSM Session Manager**, journalisé, sans clé SSH.

CloudFront et ALB

- CloudFront ajoute un en-tête secret (`X-Origine-Shopimt` , valeur aléatoire générée par Terraform). L'ALB transmet aux serveurs seulement les requêtes qui le portent ; les autres reçoivent 403. Avec la liste de préfixes CloudFront dans le groupe de sécurité, l'ALB est inaccessible en direct.
- Cache : `CachingOptimized` pour `/assets/*` (noms avec empreinte Vite) et `/img/*` , `CachingDisabled` pour le reste (API et pages).
- Groupe cible : contrôle `/sante` toutes les 15 s, 2 succès pour entrer, 3 échecs pour sortir. `/sante` exécute `SELECT 1` sur RDS : un serveur qui n'atteint plus la base est retiré du trafic.

Auto Scaling

- Modèle de lancement : dernière AMI Amazon Linux 2023 (paramètre public SSM), t3.micro, IMDSv2 obligatoire, disque gp3 8 Go chiffré, crédits CPU `standard` (aucun surcoût).
- Groupe : minimum 1, maximum 2, sur les sous-réseaux des deux zones, contrôle de santé **ELB** (une instance jugée malade par l'ALB est remplacée).
- Politique de suivi de cible : CPU moyen 60 %. Au-delà, une 2e instance démarre ; quand la charge retombe, elle est supprimée.
- **Instance refresh** : chaque nouvelle version du code (empreinte du zip dans le nom de l'objet S3) crée une version du modèle et remplace les instances une par une, la nouvelle étant saine avant l'arrêt de l'ancienne.

Serveur de traitement et Kafka

- Instance unique à IP privée fixe, pour que les serveurs web trouvent le broker sans service de découverte.
- **Récupération automatique EC2** : une alarme CloudWatch sur `StatusCheckFailed_System` relance l'instance sur un autre hôte physique, avec la même IP et le même disque.

- Kafka 3.9.1 en mode **KRaft** (un nœud broker et contrôleur, sans ZooKeeper), mémoire limitée à 320 Mo, 1 Go de swap. L'archive de Kafka est mise en cache dans S3 au premier démarrage.
- Topic `commandes`, 3 partitions. La clé est le numéro de commande : tous les événements d'une commande restent dans l'ordre dans la même partition.
- Deux groupes de consommateurs lisent tout le flux, chacun avec ses propres offsets, validés après traitement (au moins une fois).
- Si Kafka est arrêté, une commande est quand même acceptée et traitée (SNS et SQS suffisent) ; l'envoi Kafka abandonne au bout de 2 s et le journal indique `KAFKA_INDISPONIBLE`.
- Un balayage RDS toutes les minutes rattrape une commande restée `RECUE` plus de 2 minutes (par exemple si SNS a échoué juste après le `COMMIT`).

Base RDS

Table	Colonnes principales	Rôle
produits	id, nom, nom_en, prix, stock (contrainte `stock >= 0`)	Catalogue, créé au premier démarrage
commandes	numero (AUTO_INCREMENT), order_id, client_email, langue, total, statut, articles (JSON), transporteur, numero_suivi, cree_le, livraison_estimee	Commandes
historique	order_id, horodatage, statut, détail	Une ligne par étape

- MySQL 8.4 (version LTS ; 8.0 est passé en support étendu payant).
- `require_secure_transport = 1` : connexion TLS obligatoire, les serveurs vérifient le certificat RDS.
- Mot de passe de 32 caractères généré par Terraform, rangé dans SSM Parameter Store (SecureString, clé KMS gérée `aws/ssm`). Les serveurs le lisent au démarrage avec leur rôle IAM ; il n'apparaît ni dans le code ni dans le user_data.
- Sauvegarde automatique quotidienne, restauration à la seconde près sur 24 h. Stockage chiffré, 20 Go sans extension automatique.

Démarrage d'un serveur (user_data)

1. Swap de 1 Go, installation de Python 3.12 et de l'agent CloudWatch (et de Java 17 sur le serveur de traitement).

2. Téléchargement du code (et du front pour les serveurs web) depuis S3, par l'endpoint Gateway du VPC.
3. Bundle de certificats RDS, environnement virtuel Python et dépendances (`flask` , `gunicorn` , `pymysql` , `kafka-python` , `boto3`).
4. Service systemd `shopimt` démarré tout de suite : les commandes sont traitées même avant que Kafka soit prêt.
5. Sur le serveur de traitement : Kafka (depuis le cache S3), formatage du stockage KRaft, service `kafka` , création du topic.
6. Agent CloudWatch (journaux et mémoire).

Rôles IAM

Rôle	Droits propres	Communs
web	`sns:Publish` sur le topic	lecture de `code/*` dans S3, lecture du paramètre du mot de passe, Session Manager, agent CloudWatch
traitement	lecture et suppression dans la file SQS, `s3:PutObject` sur `audit/*` et `kafka/*` , envoi SES	idem

Haute disponibilité et résilience

Panne	Ce qui se passe
Un serveur web tombe	L'ALB le retire en 45 s au plus, l'Auto Scaling en lance un autre (dans l'autre zone si besoin)
Une zone de disponibilité tombe	L'ALB n'envoie plus de trafic vers elle, l'Auto Scaling relance dans l'autre zone
Pic de charge	2e serveur web au-delà de 60 % de CPU
Serveur de traitement arrêté	Les commandes sont acceptées et attendent dans SQS (14 jours), puis sont traitées au redémarrage
Panne matérielle du serveur de traitement	Récupération automatique EC2, même IP et même disque

Panne	Ce qui se passe
Broker Kafka arrêté	Commandes et e-mails de confirmation continuent ; seuls les événements Kafka de cette période manquent
RDS	Mono-AZ (gratuit) : la sauvegarde permet la restauration. En production, Multi-AZ (variable `bdd_multi_az`) bascule en 1 à 2 minutes sur l'instance de secours

Coût

Le compte est encore dans l'offre gratuite de 12 mois (750 h par mois d'EC2 t3.micro, de RDS db.t4g.micro, d'ALB et d'adresse IPv4 publique).

Élément	Gratuit	Au-delà
2 instances t3.micro 24 h/24	750 h sur 1 440 h	~0,27 $/jour
RDS db.t4g.micro, 20 Go	750 h et 20 Go	0
ALB	750 h et 15 LCU	0
4 IPv4 publiques (2 ALB, 2 instances)	750 h sur 2 880 h	~0,35 $/jour
CloudFront, SNS, SQS, SES, S3, CloudWatch	offres permanentes	0

Total : environ **0,60 $ par jour** quand la solution EC2 tourne, rien quand elle est désactivée. Volontairement exclus : NAT Gateway (~1 $/jour), Amazon MSK (~2 $/jour pour un cluster minimal), RDS Multi-AZ, 2e serveur web permanent. Ce seraient les choix d'une vraie production.

Comparaison des deux solutions

	Serverless	EC2
Calcul	Lambda (à la demande)	Serveurs t3.micro allumés en permanence
Base	DynamoDB, transaction sur le stock	RDS MySQL, `SELECT … FOR UPDATE`

	Serverless	EC2
Montée en charge	Automatique, sans réglage	Auto Scaling sur le CPU, 1 à 2 serveurs
Haute disponibilité	Services multi-zones gérés par AWS	ALB et Auto Scaling sur 2 zones, à configurer
Événements	SNS, SQS, DynamoDB Streams	SNS, SQS, Kafka
Expédition simulée	Step Functions	Fils d'exécution du serveur de traitement
Coût au repos	0	~0,60 $/jour
Exploitation	Rien à patcher	AMI, paquets, disque, mémoire à surveiller

Correspondance avec le sujet

Exigence	Réponse dans la solution EC2
Solution sur machines virtuelles	EC2 + Auto Scaling + ALB
Catalogue via une interface Web	Même front Vue, servi par les serveurs web
Commander un ou plusieurs produits	`POST /commandes`, transaction MySQL
Confirmation en temps réel	Réponse immédiate, e-mail SES en quelques secondes, page de suivi
Audit et reporting	Groupe Kafka `audit` vers S3, historique dans RDS
Emballage et expédition simulés	Serveur de traitement, mêmes étapes que le serverless
Modèle événementiel SNS	Topic SNS vers file SQS
Streaming Kafka	Broker Kafka sur EC2, deux groupes de consommateurs (MSK présenté comme cible)
Terraform	Module `ec2`, activé par une clé SSM
Haute disponibilité	2 zones, contrôle de santé, remplacement et récupération automatiques
Free Tier	Instances, RDS et ALB dans l'offre gratuite ; surcoût chiffré ci-dessus

Déployer, tester, supprimer

1. Mettre `"solution_ec2": true` dans le paramètre SSM `/shopimt/<env>/config`.
2. Déployer comme d'habitude (fusion sur `dev` ou `main`, ou `terraform apply` dans `infra/envs/<env>`). RDS prend une dizaine de minutes, puis les serveurs environ 5 minutes pour s'installer.
3. L'adresse est dans la sortie `ec2_site_url`. Passer une commande, suivre la page de suivi, ouvrir l'e-mail.
4. Dans la console : tableau de bord CloudWatch `shopimt-ec2`, groupe cible de l'ALB, fichiers `audit/` du bucket `shopimt-ec2-<compte>`, instances dans Session Manager.
5. Test de résilience : arrêter le serveur web dans la console EC2. L'ALB le retire, l'Auto Scaling en relance un, le site reste servi dès que le nouveau est sain.
6. Tests automatiques : `pytest` lance les tests de la solution EC2 contre MySQL (variable `SHOPIMT_TEST_BDD`, service MySQL 8.4 dans la CI).
7. Supprimer : `"solution_ec2": false` puis déploiement. Tout ce qui est propre à la solution EC2 est détruit, le serverless reste en ligne.

État au 30 octobre : déploiement et tests

Environnement	Site EC2	État
dev	https://dwpgpgkoyjtum.cloudfront.net	Déployé par un push sur `dev`, scénario complet réussi
prod	https://d16dvaumexekob.cloudfront.net	Déployé le 30 octobre à 11 h 33 après approbation, serveur web sain derrière l'ALB

Résultats en dev. Commandes SHOP-000001 à SHOP-000007 : stock décrémenté dans RDS, e-mails de confirmation et de suivi reçus (SES), statuts jusqu'à LIVREE (38 secondes pour la première), fichiers d'audit écrits dans S3 par le groupe Kafka audit.

Deux corrections après les premiers tests :

- L'adresse de Kafka était vide au démarrage : le jeton IMDS expirait pendant l'installation des paquets. Terraform passe maintenant l'IP fixe du serveur dans le script de démarrage.
- Le premier événement Kafka après une pause était perdu sans erreur avec kafka-python 3.0.12. La version 2.3.3 est fixée dans `backend/ec2/requirements.txt` ; vérifié sur les commandes 5, 6 et 7, dont une après 7 minutes d'inactivité.

Rejouer le test : `scripts/scenario-ec2.sh dev ton.email@exemple.fr` (instances de l'Auto Scaling group, ALB fermé, catalogue, erreurs 400 et 409, commande, suivi jusqu'à LIVREE, audit S3).